



TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 3

Problemas NP-Completos



June 15, 2026

Andres Cucchiarini
113539
Manuel Manterola
113498

Facundo Cot
113443

1 Introducción

Scaloni ya está armando la lista de 43 jugadores que van a ir al mundial 2026. Hay mucha presión por parte de la prensa para bajar línea de cuál debería ser el 11 inicial. Lo de siempre. Algunos medios quieren que juegue Roncaglia, otros quieren que juegue Mateo Messi, y así. Cada medio tiene un subconjunto de jugadores que quiere que jueguen. A Scaloni esto no le importa, no va a dejar que la prensa lo condicione, pero tiene jugadores jóvenes a los que esto puede afectarles.

Justo hay un partido amistoso contra Burkina Faso la semana que viene. Oportunidad ideal para poner un equipo que contente a todos, baje la presión y poder aislar al equipo.

El problema es, ¿cómo elegir el conjunto de jugadores que jueguen ese partido (entre titulares y suplentes que vayan a entrar)? Además, Scaloni quiere poder usar ese partido para probar cosas aparte. No puede gastar el amistoso para contentar a un periodista mufa que habla mal de Messi, por ejemplo. Quiere definir el conjunto más pequeño de jugadores necesarios para contentarlos y poder seguir con la suya. Con elegir un jugador que contente a cada periodista/medio, le es suficiente.

Ante este problema, Bilardo se sentó con Scaloni para explicarle que en realidad este es un problema conocido (viejo zorro como es, ya se comió todas las operetas de prensa así que se conoce este problema de memoria). Se sirvió una copa de Gatorei y le comentó:

“Esto no es más que un caso particular del Hitting-Set Problem. El cual es: Dado un conjunto A de n elementos y m subconjuntos $B_1, B_2, \dots, B_m$ de:

$$A (B_i \subseteq A \forall i) \tag{1}$$

Queremos el subconjunto C perteneciente a A de menor tamaño tal que C tenga al menos un elemento de cada B_i . En nuestro caso, A son los jugadores convocados, los B_i son los deseos de la prensa, y C es el conjunto de jugadores que deberían jugar contra Burkina Faso si o si”.

Bueno, ahora con un poco más claridad en el tema, Scaloni necesita de nuestra ayuda para ver si obtener este subconjunto se puede hacer de forma eficiente (polinomial) o, si no queda otra, con qué alternativas contamos.

2 Respuestas

2.1 Consigna 1

Demostrar que el Hitting-Set Problem se encuentra en NP.

Para demostrar que Hitting Set pertenece a NP, basta con mostrar que, dada una solución candidata, esta puede verificarse en tiempo polinomial.

En este caso, la solución candidata es un subconjunto:

$$C \subseteq A \quad (2)$$

Ese C sería la supuesta solución. Y debemos pasar a verificar las siguientes cosas para esto:

1. Que C sea subconjunto de A : Que todos los elementos elegidos en C pertenezcan realmente al conjunto original A .

2. Que C tenga tamaño a lo sumo k

$$|C| \leq k \quad (3)$$

3. Que C toque a todos los subconjuntos B_i Para cada elemento B_i verificamos que haya al menos un elemento en común con C

$$C \cap B_i \neq \emptyset \quad \forall i \quad (4)$$

Ahora debemos de ver si esto es polinomial, pero no de la solución desde cero, solo estamos verificamos una solución propuesta. Obteniendo que en el peor de los casos, en el subconjunto B_i , podemos recorrer sus elementos y chequear si alguno está en C .

Si usamos una representación simple, esto puede costar algo como:

$$O(m.n)$$

Porque hay m subconjuntos y cada uno puede tener hasta n elementos. Esto es polinomial en el tamaño de entrada, por lo tanto existe un verificador polinomial tal que el problema de Hitting-Set en su versión de decisión se encuentra en NP.

2.2 Consigna 2

Demostrar que el Hitting-Set Problem es, en efecto, un problema NP-Completo.

Para demostrar que Hitting Set es NP-Completo, voy a usar el problema de Vertex Cover en su versión de decisión.

Vertex Cover decisión: dado un grafo $G = (V, E)$ y un número K , ¿existe algún subconjunto de a lo sumo K vértices tal que toda arista del grafo tenga al menos una de sus puntas en el conjunto?

Considerando también el problema de Hitting Set en su versión de decisión, voy a plantear la reducción:

$$Vertex\ Cover \leq Hitting\ Set$$

Para eso, planteo la siguiente transformación. Dado un grafo $G = (V, E)$, creo una instancia de Hitting Set donde el conjunto de elementos será:

$$A = V$$

Luego, por cada arista $e = (u, v) \in E$, creo un subconjunto B_e , formado por los dos vértices que conecta esa arista:

$$B_e = \{u, v\}$$

Es decir, voy a tener un subconjunto por cada arista del grafo. Además, mantengo el mismo valor de K .

Con esta transformación voy a demostrar que hay un Vertex Cover de a lo sumo K vértices si y solo si hay un Hitting Set de a lo sumo K elementos.

Ida: Vertex Cover $\rightarrow$ Hitting Set

Si tengo una solución a Vertex Cover de a lo sumo K vértices, entonces todas las aristas del grafo tienen al menos una punta dentro de la solución.

Como en la instancia de Hitting Set hay un subconjunto por cada arista, y cada subconjunto contiene los dos vértices que conecta dicha arista, entonces la solución de Vertex Cover tendrá al menos un elemento en común con cada subconjunto B_e .

Por lo tanto, ese mismo conjunto de vértices también es una solución válida para Hitting Set de tamaño a lo sumo K , ya que intersecta a todos los subconjuntos. Si algún subconjunto no tuviera ningún elemento en común con la solución, entonces la arista correspondiente no estaría cubierta, contradiciendo que era un Vertex Cover válido.

Vuelta: Hitting Set $\rightarrow$ Vertex Cover

Si tengo una solución a Hitting Set de a lo sumo K elementos, entonces esa solución tiene al menos un elemento en común con cada subconjunto B_e .

Como cada subconjunto B_e representa una arista $e = (u, v)$ y contiene exactamente los dos vértices que toca esa arista, que haya al menos un elemento de B_e en la solución significa que al menos una de las dos puntas de la arista fue elegida.

Por lo tanto, todas las aristas del grafo tienen al menos una punta en la solución. Entonces ese mismo conjunto es un Vertex Cover de tamaño a lo sumo K .

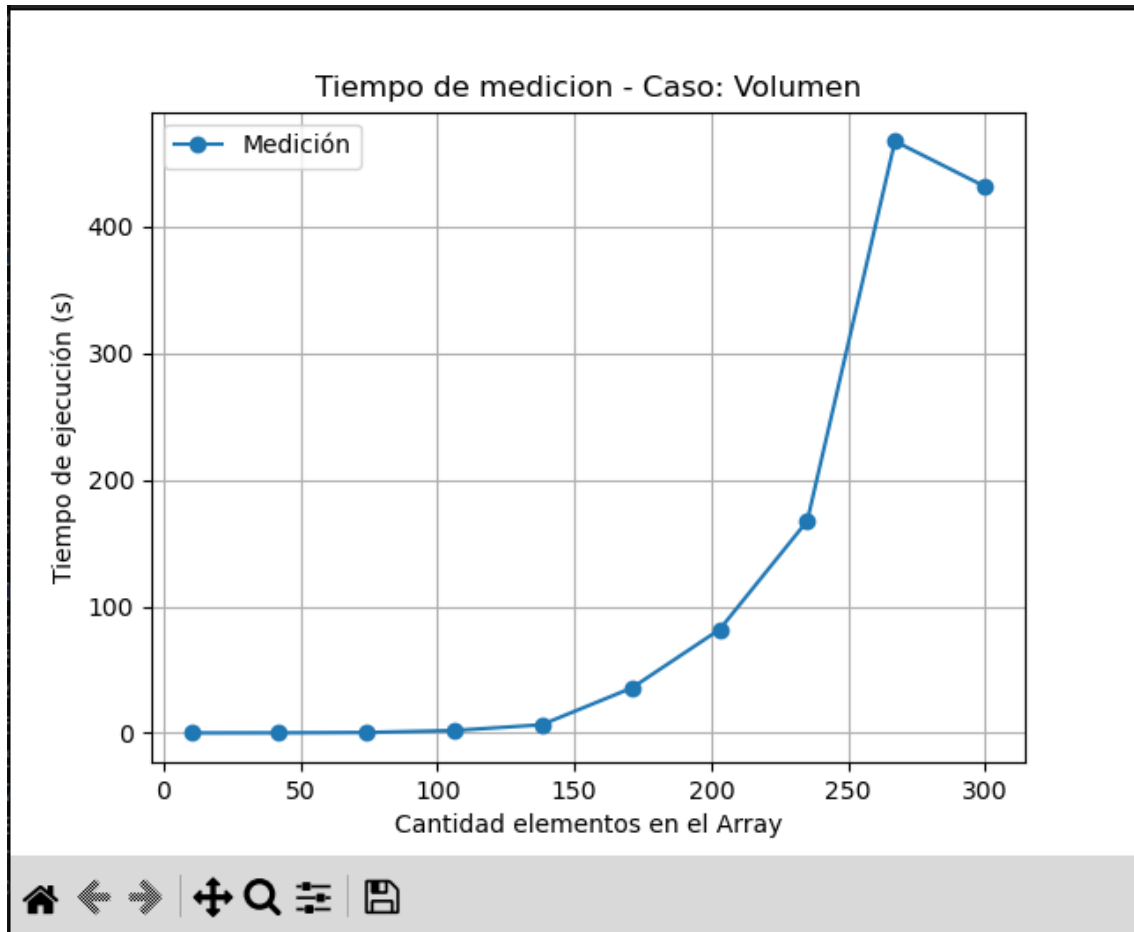
Con esto queda demostrado que existe un Vertex Cover de tamaño a lo sumo K si y solo si existe un Hitting Set de tamaño a lo sumo K . Además, la transformación se puede hacer en tiempo polinomial, ya que solamente se recorre cada arista una vez para crear su subconjunto correspondiente.

Como Vertex Cover es NP-Completo y se reduce polinomialmente a Hitting Set, y como demostramos previamente que Hitting Set pertenece a NP, podemos concluir que Hitting Set pertenece a NP-Completo

2.3 Consigna 3

Escribir un algoritmo que, por backtracking, obtenga la solución óptima al problema. Generar sets de datos para corroborar su correctitud, así como tomar mediciones de tiempos.

A continuacion se deja la imagen de los tiempos de una medicion de volumen en backtracking:



Si bien se generaron diferentes mediciones sobre distintos casos para medir y analizar el comportamiento del algoritmo de bt, el grafico de la prueba de volumen nos aporta una amplia informacion sobre el mismo. Los sets de datos se generaron utilizando el modulo `random` de python. Cada uno de ellos suele contar con una cantidad fija de periodistas de entre 2 y 6 jugadores. Ahora bien, si miramos con atencion el gráfico, se puede observar como el algoritmo tiene una clara tendencia exponencial. Para los primeros 200 elementos el algoritmo es muy rapido y casi que constante, pero se puede ver como a raiz que se pasa la barrera de 200 elementos el tiempo que tarda incrementa muy rápidamente. También, y no menos interesante, notar que entre 250 y 300 jugadores hubo una reduccion en el tiempo, uno podria preguntarse ¿Cómo es posible que con mas elementos tarde menos?, la respuesta es simple. Se debe a que en bt no solo importa la cantidad de elementos si no el orden natural de los mismos. Es decir, si bien la tendencia es exponencial, puede que ser que en por ejemplo, si tenemos un periodista con una lista de 100 jugadores y otra lista con 200, si el algoritmo tiene que evaluar 50 jugadores de la primer lista para encontrar el que le conviene y luego va y en la otra el primero ya es válido, pues entonces tardo mucho menos y se encontro mucho mas rápido, si bien esto no hace mas que bajarnos un poco el tiempo final, la complejidad teorica sigue siendo exponencial.

COMPLEJIDAD:

Siendo n la cantidad de maxima jugadores que pide un periodista y m la cantidad de periodistas. La complejidad resulta en $O(n^m)$. Dado que por cada periodista se abren n cantidades

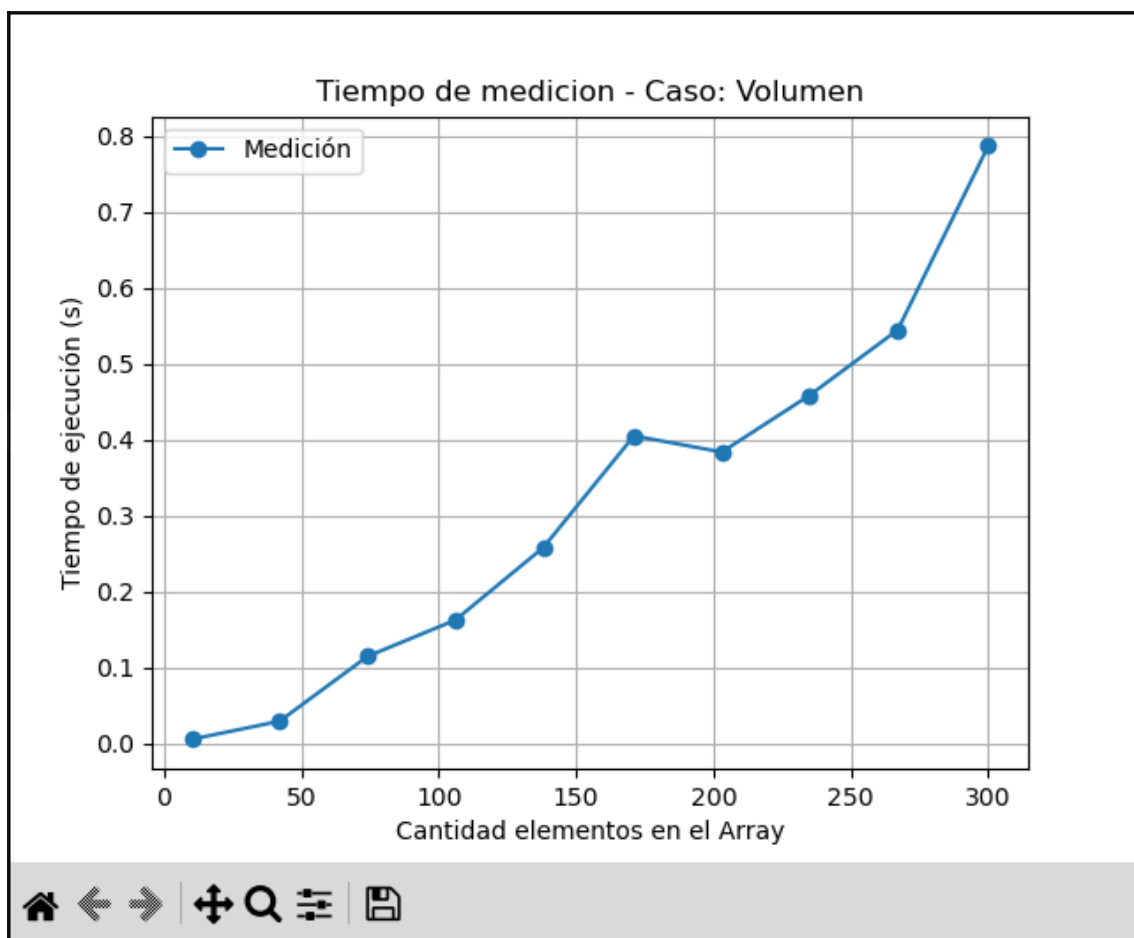
de opciones, y se expande en profundidad tantas veces como periodistas haya. Esto nos da la complejidad exponencial que fue vista en el gráfico mas arriba.

2.4 Consigna 4

Escribir un modelo de programación lineal que resuelva el problema de forma óptima. Ejecutarlo para los mismos sets de datos para corroborar su correctitud (o al menos hasta la mayor cantidad de volumen posible). Tomar mediciones de tiempos y compararlas con las del algoritmo que implementa Backtracking.

$$\min \sum_{j=1}^n x_j$$

$$\text{s.a.} \quad \sum_{a_j \in B_i} x_j \geq 1 \quad \forall i = 1, 2, \dots, m$$
$$x_j \in \{0, 1\} \forall j = 1, 2, \dots, n$$



Aca se puede observar el gráfico a la misma prueba de volumen que fue ejecutada para bt. Como conclusion de esto, se puede decir que el algoritmo de pl tiene una tendencia exponencial también, pero mucho menor a la del algoritmo de bt, esto se debe a que dada nuestra solución a partir de variables enteras, se obtiene como complejidad $O(2^n)$ (siendo n la cantidad de jugadores). Es decir, sacando el caso donde los periodistas tengan constantemente 2 jugadores cada uno, este algoritmo va a ser mucho más eficiente que el de bt, ya que cuanto más jugadores tengan en la lista los futbolistas, más acelerado va a ser el crecimiento de la curva debido a que la base es mucho mayor.

2.5 Consigna 5

El doctor Bilardo, como buen resultadista que es, le propone a Scaloni el siguiente algoritmo: usar el mismo algoritmo planteado en el punto 4, pero permitiendo que las variables de decisión sean valores reales, y luego redondear el resultado final del modelo. Para redondear, obtenemos el valor b como la cantidad de aquel conjunto entre los diferentes conjuntos pedidos de la prensa que tenga la mayor cantidad de jugadores, y definimos que la variable de decisión de cada jugador será 1 si su valor en el modelo relajado es mayor o igual a $1/b$.

Este algoritmo sirve como una aproximación para resolver el Hitting-Set Problem. Implementar dicho algoritmo, analizar su complejidad y analizar cuán buena aproximación es. Para esto, considerar lo siguiente: sea I una instancia cualquiera del Hitting-Set Problem, y $z(I)$ una solución óptima para dicha instancia, y sea $A(I)$ la solución aproximada, se define:

$$\frac{A(I)}{z(I)} \leq r(A) \quad (5)$$

para todas las instancias posibles. Calcular $r(A)$ para el algoritmo dado, demostrando que la cota está bien calculada. Realizar mediciones utilizando el algoritmo exacto y la aproximación, con el objetivo de verificar dicha relación. Realizar también mediciones que contemplen volúmenes de datos ya inmanejables para el algoritmo exacto, a fin de corroborar empíricamente la cota calculada anteriormente.

El algoritmo relajado sería este, con n perteneciente a todos los elementos de A

$$\min \sum_{j=1}^n x_j$$

$$\begin{aligned} \text{s.a.} \quad & \sum_{a_j \in B_i} x_j \geq 1 \quad \forall i = 1, 2, \dots, m \\ & 0 \leq x_j \leq 1 \quad \forall j = 1, 2, \dots, n \end{aligned}$$

$$SL = \sum_{j=1}^n x_j$$

Ahora para ver con qué elementos de a me quedo defino a b como:

$$b = \max_{1 \leq i \leq m} |B_i|$$

Siendo m la cantidad de subconjuntos. Y la solución a mi algoritmo van a ser todos los a_j tales que:

$$C = \left\{ a_j \in A : x_j \geq \frac{1}{b} \right\}$$

Primero demostramos que el algoritmo da una solución válida: Para eso primero vamos a tener en cuenta que nuestro algoritmo lineal restringe que:

$$\sum_{a_j \in B_i} x_j \geq 1.$$

Si en nuestro algoritmo tenemos algún subconjunto B_i de tal manera que todos los x_j son:

$$x_j < \frac{1}{b}.$$

Entonces se cumple que para ese subconjunto:

$$\sum_{a_j \in B_i} x_j < \sum_{a_j \in B_i} \frac{1}{b}.$$

Como el subconjunto B_i tiene $\text{len}(B_i)$ elementos es lo mismo decir que:

$$\sum_{a_j \in B_i} x_j < \frac{|B_i|}{b}.$$

Y como sabemos que $\text{len}(B_i) \leq b$ podemos decir que:

$$\frac{|B_i|}{b} \leq 1.$$

Entonces me queda que:

$$\sum_{a_j \in B_i} x_j < 1.$$

Pero esto contradice la condicion inicial y principal de mi SL:

$$\sum_{a_j \in B_i} x_j \geq 1.$$

Por lo tanto no puede existir ningun suconjunto que no tenga al menos algun elemento en C .

Habiendo demostrado que mi modelo si o si encuentra soluciones validas ahora hay que calcularle la cota de aproximacion:

Sea OPT el valor encontrado por la solucion lineal entera, Y sea SL el valor encontrado por el algoritmo lineal no entero, se va a cumplir si o si que

$$SL \leq OPT.$$

Ya que el algoritmo no entero puede encontrar la solucion optima pero tambien puede encontrar mejores en su version no entera fraccionando valores.

Tambien vamos a usar que para todos los valores que estan en C :

$$x_j \geq \frac{1}{b}.$$

Entonces $1 \leq bx_j$. Podemos decir que $|C|$ es la sumatoria de todos los subconjuntos que pertenecen a C :

$$|C| = \sum_{a_j \in C} 1.$$

Y con la desigualdad demostrada arriba nos queda que:

$$|C| = \sum_{a_j \in C} 1 \leq \sum_{a_j \in C} bx_j.$$

Y entonces me queda:

$$|C| \leq b \sum_{a_j \in C} x_j.$$

Nosotros sabemos que sumar a todos los elementos de C es menor o igual a sumar a todos los elementos de A

$$\sum_{a_j \in C} x_j \leq \sum_{j=1}^n x_j.$$

Entonces usando que:

$$|C| = \sum_{a_j \in C} 1 \leq \sum_{a_j \in C} bx_j.$$

Podemos decir que:

$$|C| \leq b \sum_{j=1}^n x_j.$$

Como:

$$SL = \sum_{j=1}^n x_j,$$

Entonces:

$$|C| \leq b \cdot SL.$$

Y como $SL \leq OPT$ puedo decir que:

$$|C| \leq b \cdot OPT.$$

Por lo tanto:

$$\frac{|C|}{OPT} \leq b.$$

Y gracias a esto podemos decir que $r(A) = b$, entonces nuestro algoritmo es una b -aproximación, siendo b el tamaño de el subconjunto que mas elementos tiene.

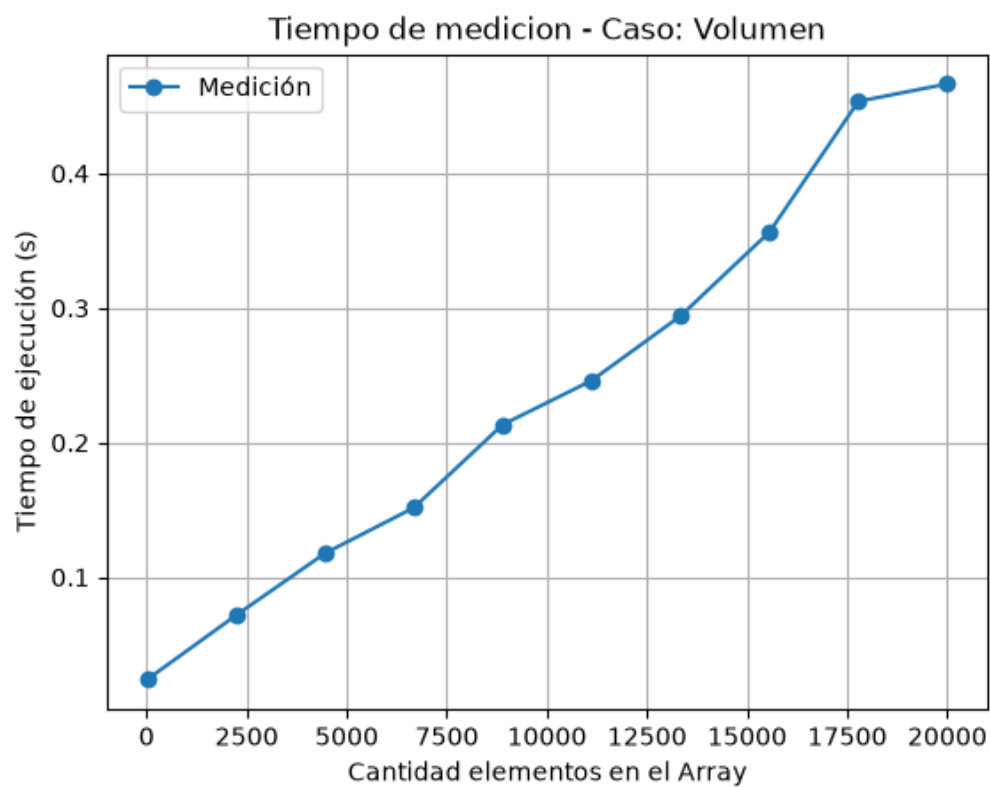
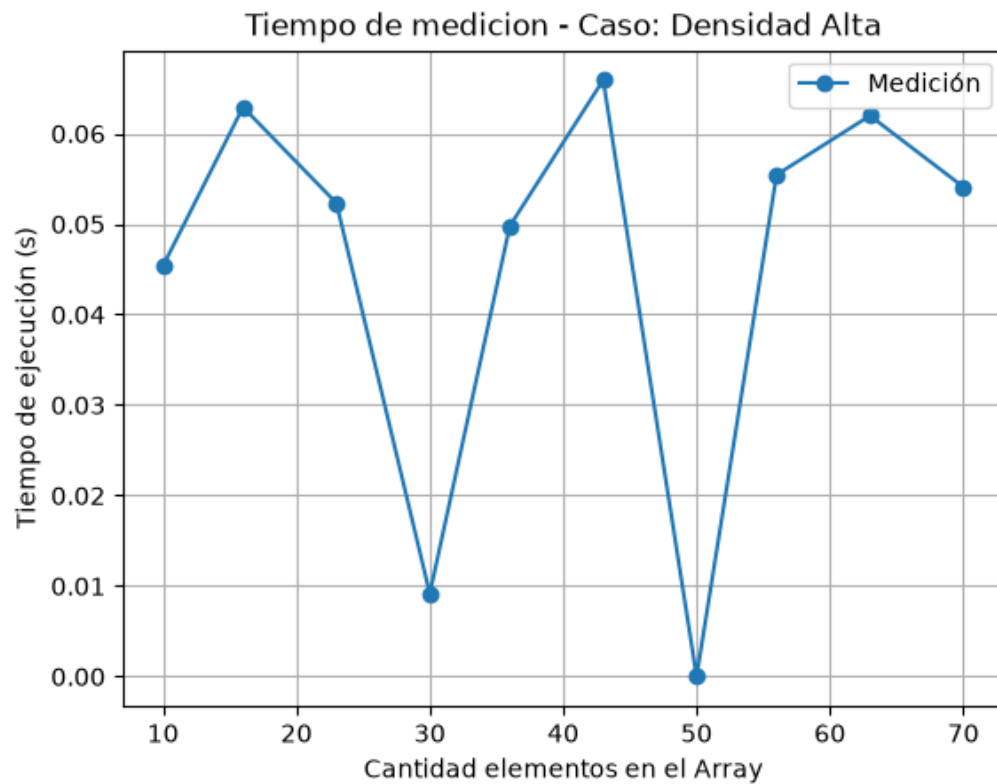
Pruebas

Para verificar esto se realizaron diversas mediciones para chequear los distintos casos que pueden ocurrir en el problema, lo primero fue ver que la solución del problema efectivamente no era mayor que una b -aproximación. Para esto se compararon a través de distintos archivos las soluciones que daban el PL óptimo y la aproximación, en todos los casos comparados se cumplió la cota máxima, por lo tanto la demostración hecha arriba se cumple

```
OPT: 2 ["Barcon't", 'Messi']
APROX: 2 ["Barcon't", 'Messi']
b: 6
APROX / OPT: 1.0
Cumple cota: True
-----
Archivo: archivos/7.txt
OPT: 2 ['Mauro Zarate', 'Pezzella']
APROX: 2 ['Mauro Zarate', 'Pezzella']
b: 6
APROX / OPT: 1.0
Cumple cota: True
-----
Archivo: archivos/10_pocos.txt
OPT: 3 ['Casco', 'Di Maria', 'Chiquito Romero']
APROX: 6 ['Casco', 'Dibu', 'Colidio', 'Chiquito Romero', 'Cuti Romero', 'Messi']
b: 7
APROX / OPT: 2.0
Cumple cota: True
-----
Archivo: archivos/10_todos.txt
OPT: 10 ['Paredes', 'Senesi', 'Mac Allister', 'Pezella', 'Ocampos', 'Tagliafico', 'Rulli', 'Messi', 'Palacios', 'Walter Benitez']
APROX: 10 ['Senesi', 'Garnacho', 'Medina', 'Martinez Quarta', 'Mac Allister', 'Pezella', 'Tagliafico', 'Rulli', 'Messi', 'Palacios']
b: 4
APROX / OPT: 1.0
Cumple cota: True
-----
Archivo: archivos/10_varios.txt
OPT: 6 ['Dybala', 'Dibu', 'Wachoffisde Abila', 'Palermo', 'Di Maria', 'Beltran']
APROX: 6 ['Dybala', 'Dibu', 'Wachoffisde Abila', 'Palermo', 'Di Maria', 'Beltran']
b: 4
APROX / OPT: 1.0
Cumple cota: True
-----
```

Luego se realizaron mediciones de tiempo para la aproximación por programación lineal no entera. Primero se midieron los tiempos en instancias de tamaño moderado, los cuales resultaron muy rápidos

Y finalmente se realizaron mediciones para casos en los que el algoritmo óptimo sería inviable, y resultó obtener una solución en un tiempo más que aceptable.



2.6 Consigna 6

Implementar alguna otra aproximación (u algoritmo greedy) que les parezca de interés. Comparar sus resultados con los dados por la aproximación del punto 5. Indicar y justificar su complejidad. No es obligatorio hacer este punto para aprobar el trabajo práctico (pero si resta puntos no hacerlo).

El algoritmo propuesto para este Hitting Set sería partiendo de la idea del mismo donde teniendo:

$$A = a_1, a_2, \dots, a_n$$

Y además los subconjuntos:

$$B_1, B_2, \dots, B_m$$

Queremos:

$$C \subseteq A \tag{6}$$

El algoritmo planteado sería:

1. Inicializamos:

$$C = \emptyset \tag{7}$$

Y marcamos los subconjuntos B_i como no cubiertos.

2. Mientras exista algún subconjunto B_i no cubierto:

- Para cada elemento a perteneciente a A , calculamos cuántos subconjuntos no cubiertos contiene.
- Elegimos el elemento a que aparece en la mayor cantidad de subconjuntos no cubiertos y agregamos ese elemento a la solución:

$$C = C \cup \{a\} \tag{8}$$

- Marcamos como cubiertos todos los subconjuntos B_i tales que:

$$a \in B_i \tag{9}$$

3. Cuando todos los subconjuntos están cubiertos, devolvemos C .

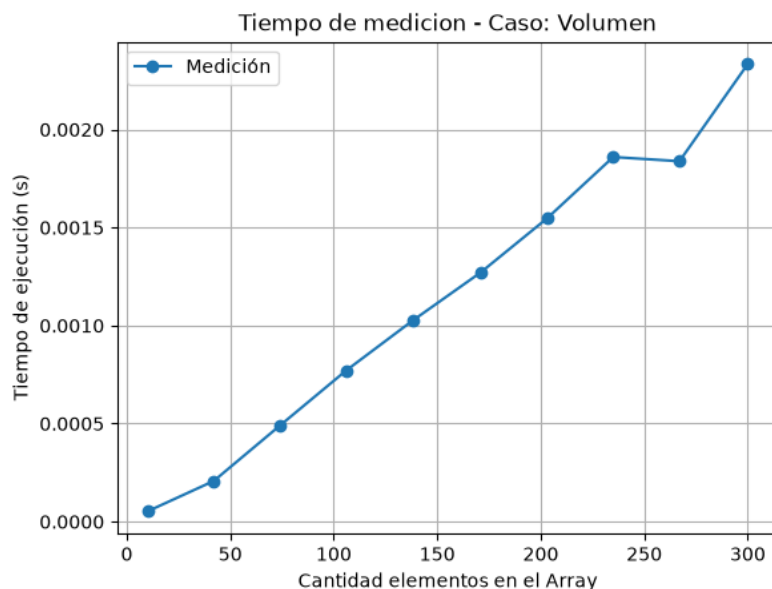
```

1 def greedy_hitting_set(A, subconjuntos):
2     C = set()
3     no_cubiertos = []
4     for B in subconjuntos:
5         no_cubiertos.append(B)
6
7     while len(no_cubiertos) > 0:
8         mejor_elemento = None
9         mejor_cantidad = -1
10
11        for elemento in A:
12            cantidad = 0
13
14            for B in no_cubiertos:
15                if elemento in B:
16                    cantidad += 1
17
18            if cantidad > mejor_cantidad:
19                mejor_cantidad = cantidad
20                mejor_elemento = elemento
21
22        C.add(mejor_elemento)
23        nuevos_no_cubiertos = []
24
25        for B in no_cubiertos:
26            if mejor_elemento not in B:
27                nuevos_no_cubiertos.append(B)
28
29        no_cubiertos = nuevos_no_cubiertos
30
31    return C

```

La complejidad de este algoritmo Greedy es de $O(n^2 * m)$, donde $n = |A|$ y m es la cantidad de subconjuntos. Esto se debe a que, en cada iteración, se recorren los n elementos de A , y para cada uno se verifica en cuántos de los m subconjuntos aún no cubiertos aparece.

Se realizaron mediciones del algoritmo greedy utilizando la misma metodología empleada para la aproximación por programación lineal. En ambos casos se utilizó la instancia de tipo Volumen, manteniendo el mismo generador de datos y el mismo procedimiento de medición. La única diferencia fue el algoritmo evaluado, en un caso se midió la aproximación basada en programación lineal y, en el otro, el algoritmo greedy propuesto.



Al comparar los gráficos obtenidos para el caso Volumen, se observa que el algoritmo greedy presenta tiempos de ejecución considerablemente menores. Mientras que la aproximación por programación lineal alcanza tiempos del orden de décimas de segundo, el greedy se mantiene en el orden de milésimas de segundo. Esto se debe a que el greedy no resuelve un modelo de programación lineal, sino que en cada iteración cuenta apariciones de jugadores en los subconjuntos no cubiertos y selecciona aquel que cubre la mayor cantidad de pedidos pendientes.

Por lo tanto, para instancias de mayor volumen, el algoritmo greedy resulta más eficiente computacionalmente. Sin embargo, esta mejora en tiempo no implica necesariamente una mejor calidad de solución, ya que el greedy toma decisiones locales y no garantiza obtener el subconjunto de menor tamaño.